gph2cgns 坐标解析 Bug 修复开发总结

1. 问题背景

`gph2cgns` 工具用于将 SCTpre 生成的专有二进制格式 `.gph` 文件转换为工业标准 CFD 网格格式 CGNS/HDF5。测试发现输出的坐标值严重错误：`CoordinateX` 应在 `0-0.01` 范围内，实际解析结果却出现大量 **89000+** 的异常值，显然是解析错位所致。

2. 问题现象

项目	期望值	实际值
顶点数	52	108（计算错误）
CoordinateX 范围	[0, 0.01]	[-683142528, 3.5e34]
CoordinateY 范围	[0, 0.01]	[-1.5e29, 7.9e34]
CoordinateZ 范围	[0, 0.01]	[-4.3e15, 4.2e36]
含 89000+ 的坐标分量数	0	22

运行原始代码的具体症状：

```
Vertex 0: (0.000000, 0.000000, 1.035000)   ← z 看似合理，实为误读
Vertex 1: (89128.960938, 0.996722, -0.000000)  ← X 出现 89000+
Vertex 2: (1.035000, 89128.960938, 1.035000)   ← Y 出现 89000+
```

3. 调查过程

3.1 排查硬编码偏移量

原代码 `parse_gph_mesh()` 使用以下硬编码参数读取坐标：

```python
# 原代码（错误）
nodes_section_end = 0x2C60
nodes_data_start  = 0x2750
n_vertices = (nodes_section_end - nodes_data_start) // 12  # = 108（错误）

for i in range(n_vertices):
    base = nodes_data_start + i * 12
    xyz[i, 0] = read_f32_be(data, base)      # float32
    xyz[i, 1] = read_f32_be(data, base + 4)  # float32
    xyz[i, 2] = read_f32_be(data, base + 8)  # float32
```

初步判断为"偏移量错误"，但调整偏移量和步长后，89000+ 问题依然存在。

3.2 定位 89000+ 的来源

在文件中扫描所有 float32 > 80000 的位置，发现异常值 `89128.96` 对应的 4 字节为 `47 AE 14 7B` ，集中出现在 `LS_Nodes` 节（偏移 `0x26B0-0x2C60` ）内部，且呈现规律性分布。

3.3 关键发现——坐标实为 float64

注意到同一区域另一个高频值 `1.035` ，对应 4 字节 `3F 84 7A E1` 。将两者拼接为 8 字节后发现：

```
字节序列: 3F 84 7A E1  47 AE 14 7B
↓ 标准大端 float64 解释
0x3F847AE147AE147B  =  0.01（精确值）
```

关键验证：

```
import struct
val = struct.unpack(">d", bytes([0x3F,0x84,0x7A,0xE1,0x47,0xAE,0x14,0x7B]))[0]
# val = 0.01000000000000000021  ← 正是期望的坐标最大值！

struct.pack('>d', 0.01).hex()
# '3f847ae147ae147b'  ← 与文件中两个"异常值"拼合完全一致
```

这说明 `(1.035, 89128.96)` 不是两个独立的 float32，而是同一个 float64 值 `0.01` 被**错误截断**的产物：

读法	字节	结果
float32（错误）	`3F 84 7A E1`	**1.035**
float32（错误）	`47 AE 14 7B`	**89128.96**
float64（正确）	`3F 84 7A E1 47 AE 14 7B`	**0.01** ✓

3.4 发现字反转存储方式

进一步分析发现文件中坐标 `0.01` 并非以标准大端 float64 存储，而是以**字反转（word-reversed / middle-endian）**格式存储：

```
标准大端 float64 (0.01):    3F 84 7A E1  47 AE 14 7B
                             [  高32位   ][  低32位   ]

文件中实际字节顺序:         47 AE 14 7B  3F 84 7A E1
                             [  低32位   ][  高32位   ]
```

即每个 8 字节 double 的两个 32 位字顺序被调换，但每个字内部仍保持大端序（类似 PDP-endian / 混合端序）。

3.5 揭示 LS_Nodes 节的三轴块结构

通过对节内描述符的系统解析，确认 `LS_Nodes` 节将三个坐标轴分别以独立数据块存储，块间以描述符隔开：

```
LS_Nodes 节结构（0x26B0–0x2C60）
├── 40B  节标签头 [I4=32][名称32B][I4=32]
├── 4×16B 元数据描述符
│
├── [16B 描述符: 12/8/52/1]   ← X 轴块入口
├── [12B 块头:  12/416/1.035] ← byte_count=416, max_X 高32位
├── 52×8B X 坐标（字反转 float64）
│
├── [16B 描述符: 12/8/52/1]   ← Z 轴块入口
├── [12B 块头:  12/416/1.035]
├── 52×8B Z 坐标（字反转 float64）
│
├── [16B 描述符: 12/8/52/1]   ← Y 轴块入口
├── [12B 块头:  12/416/0.0  ]
└── 52×8B Y 坐标（字反转 float64）
```

4. 根本原因总结

原代码存在三个层叠错误：

#	错误	原始值	正确值
1	数据类型	float32（4 字节）	float64（8 字节，字反转）
2	读取起始偏移	硬编码 `0x2750`	动态定位第一个 `[12/8/n/1]` 描述符后的数据块
3	顶点计数	`(0x2C60-0x2750)//12 = 108`	从描述符读取 `dim0 = 52`

错误 1 是产生 89000+ 异常值的直接原因：将坐标值 `0.01` 的低32位 `0x47AE147B` 误读为 float32，得到 `89128.96` 。

5. 修复方案

5.1 新增字反转 float64 读取函数

```python
def read_f64_wr(data: bytes, pos: int) -> float:
    """读取字反转 float64：文件存储顺序为 [低32位大端][高32位大端]。"""
    lower = int.from_bytes(data[pos : pos + 4], "big")
    upper = int.from_bytes(data[pos + 4 : pos + 8], "big")
    combined = ((upper << 32) | lower).to_bytes(8, "big")
    return struct.unpack(">d", combined)[0]
```

5.2 动态节定位

```python
def _find_section(data: bytes, name: str) -> int:
    """通过节名称动态定位节起始位置（I4=32 标记处）。"""
    name_padded = name.ljust(32).encode("ascii")
    idx = data.find(name_padded)
    if idx < 4:
        return -1
    return idx - 4 if read_i32_be(data, idx - 4) == 32 else -1
```

5.3 动态解析 LS_Nodes 三轴坐标块

```python
def _parse_ls_nodes(data: bytes) -> tuple:
    sec_start = _find_section(data, "LS_Nodes")
    pos = sec_start + 40  # 跳过 40B 标签头

    # 扫描描述符，找到第一个 [12, 8, n_verts, 1]
    n_vertices = 0
    while pos + 16 <= len(data):
        if read_i32_be(data, pos) != 12:
            break
        type_code = read_i32_be(data, pos + 4)
        dim0      = read_i32_be(data, pos + 8)
        dim1      = read_i32_be(data, pos + 12)
        pos += 16
        if type_code == 8 and dim1 == 1 and dim0 > 0:
            n_vertices = dim0
            break

    # 连续读取三个轴块 (X, Z, Y)
    coord_blocks = []
    for _ in range(3):
        pos += 12                              # 跳过 12B 块头
        vals = [read_f64_wr(data, pos + i*8)
                for i in range(n_vertices)]
        pos += n_vertices * 8
        coord_blocks.append(vals)
        if pos + 16 <= len(data) and read_i32_be(data, pos) == 12:
            pos += 16                          # 跳过下一个描述符

    # 文件顺序 X,Z,Y → 输出 X,Y,Z
    x_vals, z_vals, y_vals = coord_blocks
    xyz = np.array([[x_vals[i], y_vals[i], z_vals[i]]
                    for i in range(n_vertices)], dtype=np.float64)
    return xyz, n_vertices
```

6. 修复效果对比

项目	修复前	修复后
顶点数	108（错误）	**52**（正确）
CoordinateX 范围	[-6.8e8, 3.5e34]	**[0, 0.01]**
CoordinateY 范围	[-1.5e29, 7.9e34]	**[0, 0.01]**
CoordinateZ 范围	[-4.3e15, 4.2e36]	**[0, 0.01]**
含 89000+ 的分量	22	**0**
坐标精度	float32（单精度）	**float64（双精度）**

修复后输出示例：

```
Reading: box.gph
  Vertices: 52
  Elements: 135
  Element type: HEXA_8

CoordinateX: [0.00000000, 0.01000000]  ✓
CoordinateY: [0.00000000, 0.01000000]  ✓
CoordinateZ: [0.00000000, 0.01000000]  ✓
```

7. 改动文件清单

文件	改动类型	主要内容
`gph2cgns.py`	功能修复	新增 `read_f64_wr()`、`_find_section()`、`_parse_ls_nodes()`、`_parse_ls_links()`、`_parse_n_elements()`；重写 `parse_gph_mesh()`
`gph_model.py`	同步修复	更新 LS_Nodes 节点解析逻辑，使用字反转 float64
`gph_parser.py`	文档+修复	更正节采样代码；补充字反转 float64 说明
`GPH_FORMAT_SPEC.md`	文档	更新 LS_Nodes 格式说明，新增字反转 float64 规范与 bug 复现示例

8. GPH LS_Nodes 格式规范（逆向结论）

8.1 字反转 float64（Word-Reversed Float64）

GPH 文件中的双精度浮点数采用非标准的混合端序存储：

```
IEEE 754 标准大端 (0.01):   [3F 84 7A E1] [47 AE 14 7B]
                              ↑ 高32位       ↑ 低32位

GPH 文件实际存储 (0.01):    [47 AE 14 7B] [3F 84 7A E1]
                              ↑ 低32位       ↑ 高32位
```

解码方式：

```python
lower = int.from_bytes(data[pos:pos+4], 'big')   # 先读低32位
upper = int.from_bytes(data[pos+4:pos+8], 'big') # 再读高32位
value = struct.unpack(">d", ((upper << 32) | lower).to_bytes(8, 'big'))[0]
```

8.2 LS_Nodes 块头含义

每个轴块的 12 字节头部结构：

偏移	字段	说明
+0	`I4 = 12`	本头部长度标记
+4	`I4 = byte_count`	数据负载字节数（= n_verts × 8）

偏移	字段	说明
+8	`I4`	该轴最大坐标值的**高32位**（可用于校验量级）

8.3 坐标轴存储顺序

文件内轴块顺序为 **X → Z → Y**，与 CGNS 标准轴序（X, Y, Z）不同，
读取后需重新排列。

9. 经验总结

1. **二进制格式逆向不能只靠"看起来合理"的值**。初期 float32 读取出的 1.035 坐标在 [0,1] 范围内，伪装成了合理结果，掩盖了真正错误。
2. **将 "魔法常量" 作为线索**。89128.96 (`0x47AE147B`) 在全文件中规律性出现，反而提供了关键突破口——它是 float64 坐标 0.01 的低32位的 float32 误读。
3. **数据格式规范中的"类型码"需结合上下文理解**。 `type=8` 在格式描述符中既可能是"8字节double"也可能是"其他"，需要通过实际字节验证而非仅凭规范推断。
4. **硬编码偏移量脆弱**。不同来源的 GPH 文件截面大小不同，固定偏移量 `0x2750` 只在特定测试文件中偶然正确，对通用文件必须动态解析。
5. **混合端序真实存在**。PDP-endian（字反转）虽然罕见，但确实出现在部分工业软件的历史遗留格式中，逆向时应将其纳入候选假设。